ASSEMBLY LANGUAGE

MICROPROCESSORS

Prepared by Eng. Manar Ahmed

ARM assembly

OBJECTIVES

- Recap.
- Assignment1 Solution.
- Load and Store Instructions
- Loading & Storing Bytes
- Move Instructions.
- Addressing Modes.
- Offset Addressing
- Memory Alignment
- Endianess.
- practical application.

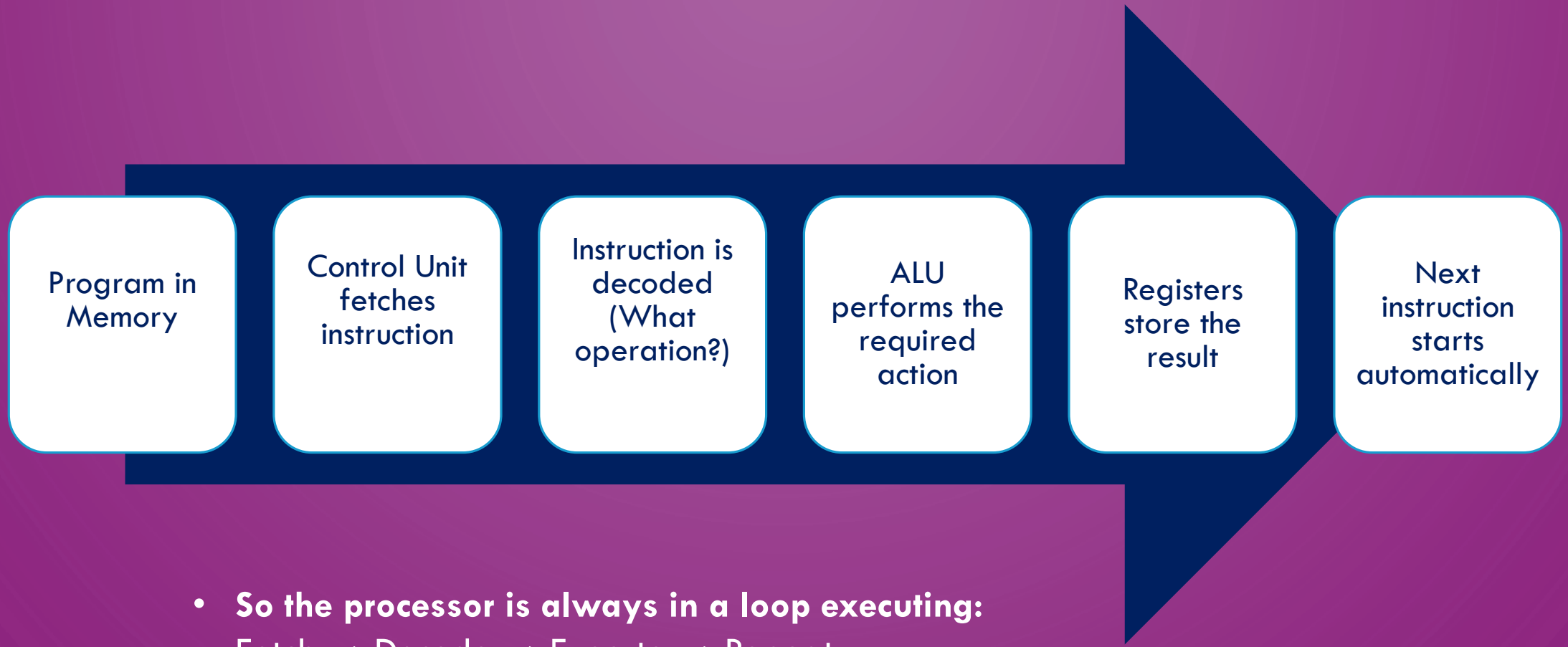
RECAP

- In the previous section:

  - All operations happened inside **Registers.**

  -The CPU cannot calculate directly from Memory.

➢ How the Microprocessor Works:



- **So the processor is always in a loop executing:**
  Fetch → Decode → Execute → Repeat

ASSIGNMENT1 SOLUTION

The Objective:

Write an ARM Assembly program to analyze a small array of numbers stored in memory. The program should calculate the **Sum** of the elements and find the **Maximum** value.

The Scenario:

1.Define an array of 5 unsigned integers in the memory (Data Section).

2.Your program must:

- **Step 1:** Load the base address of the array.
- **Step 2:** Loop through the array to calculate the total **Sum**.
- **Step 3:** During the same loop, identify the **Maximum** value in the array.
- **Step 4:** Store the final **Sum** in register R5 and the **Maximum** value in register R6.

Welcome | ass1.s

ass1.s > ...

```
1   .section .data
    2 references
2   array:      .word 5, 8, 3, 12, 7     @ Array of 5 unsigned integers
3
4   .section .text
5   .global _start
6
    1 reference
7   _start:
8       LDR R0, =array      @ Load base address of array into R0
9       MOV R1, #5          @ Number of elements in R1
10      MOV R5, #0          @ Initialize sum to 0
11      LDR R6, [R0]        @ Initialize max with first element of array
12
    1 reference
13  loop_start:
14      LDR R2, [R0], #4    @ Load current element into R2 and increment pointer
15      ADD R5, R5, R2      @ Add element to sum
16
17      CMP R2, R6           @ Compare current element with max
18      BLE skip_max_update  @ If R2 <= R6, skip
19      MOV R6, R2           @ Otherwise, update max
20
    1 reference
21  skip_max_update:
22      SUBS R1, R1, #1      @ Decrement counter
23      BNE loop_start        @ If counter not zero, repeat loop
24
25      MOV R0, R5           @ sum
26      BL print_number      @ call print function
27      MOV R0, R6           @ max
28      BL print_number      @ call print function
29
    1 reference
30  end:
31      B end                @ Infinite loop
```

RECAP

- From Registers to Memory:

- We must move data:
  Memory → Registers → Process → Back to Memory
- This is called **Data Movement.**

“ Memory access Data movement ”

WHY DATA MOVEMENT IS IMPORTANT

- Programs constantly read and write data.

- Needed for :
  - -Variables.
  - -Arrays.
  - -Sensor values.
  - -Storing results.

- ARM follows a **Load/Store Model**:
  - -Only Load/Store instructions access memory.

Load Instruction (Reading from Memory)

- Load = Copy data from Memory → Register.
- **Example:**
  LDR R0, [R1]
- Meaning:
  R0 ← Memory value stored at address inside R1.
- Used when:
  -Reading array values.
  -Reading sensor data.
  -Accessing variables.

Store Instruction (Writing to Memory)

- Store = Copy data from Register → Memory.
- **Example:**
  STR R2, [R3]
- Meaning:
  Memory[R3] ← R2
- Used when:
  -Saving results.
  -Updating variables.
  -Writing to hardware ports.

LOADING & STORING BYTES

- **LDRB** → Load **Byte** (8 bits) from memory to a register.
- **STRB** → Store **Byte** from a register to memory.

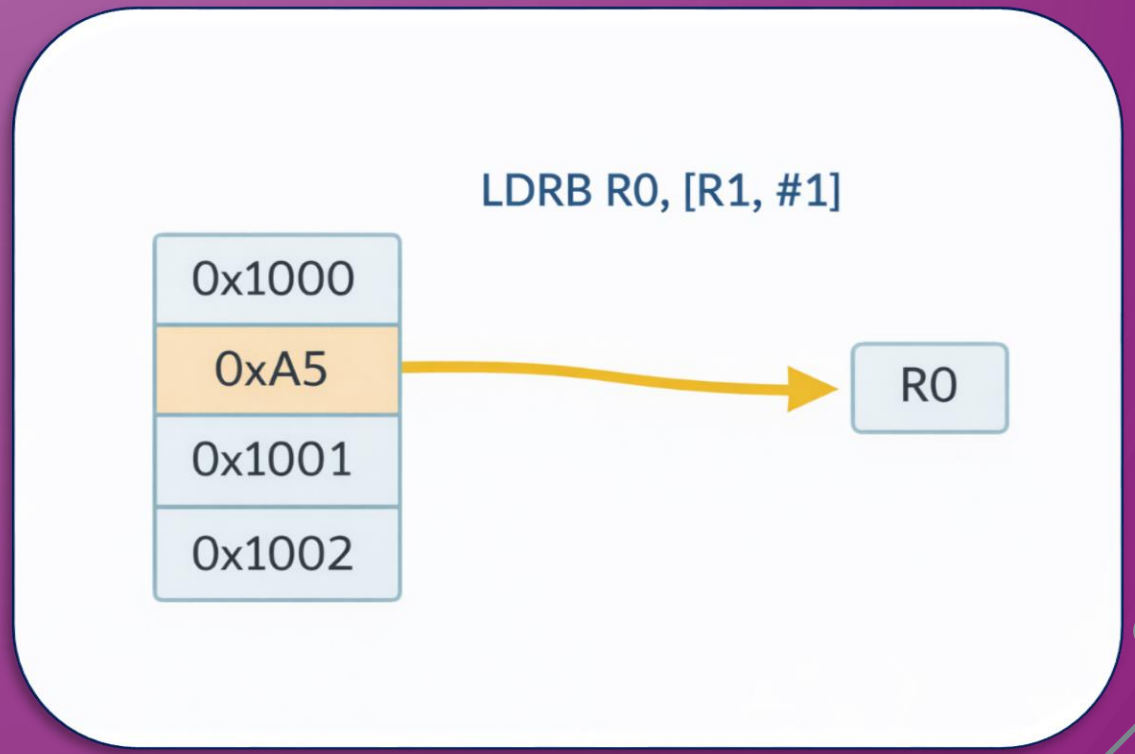
- **Example** :

- LDRB R0, [R1, #2] ;

Load 1 byte from (R1+2).

- STRB R0, [R1, #5] ;

Store 1 byte to (R1+5).



MOVE INSTRUCTION (REGISTER TO REGISTER)

- MOV does NOT access memory.
- **Example:**
  MOV R1, R0
- Meaning:
  Copy value from R0 → R1
- Why use MOV?
  -Faster than memory access.
  -Used for temporary manipulation.

Addressing Modes (How CPU Finds Data)

- Addressing Mode defines WHERE the operand comes from.
- Without addressing modes:
  The CPU would not know where the data exists.

Immediate Addressing	Register Addressing	Memory Addressing
MOV R0, #10 -Value is inside instruction.	ADD R2, R0, R1 Data already in registers..	LDR R3, [R4] Data located in memory.

OFFSET ADDRESSING IN ARM ASSEMBLY

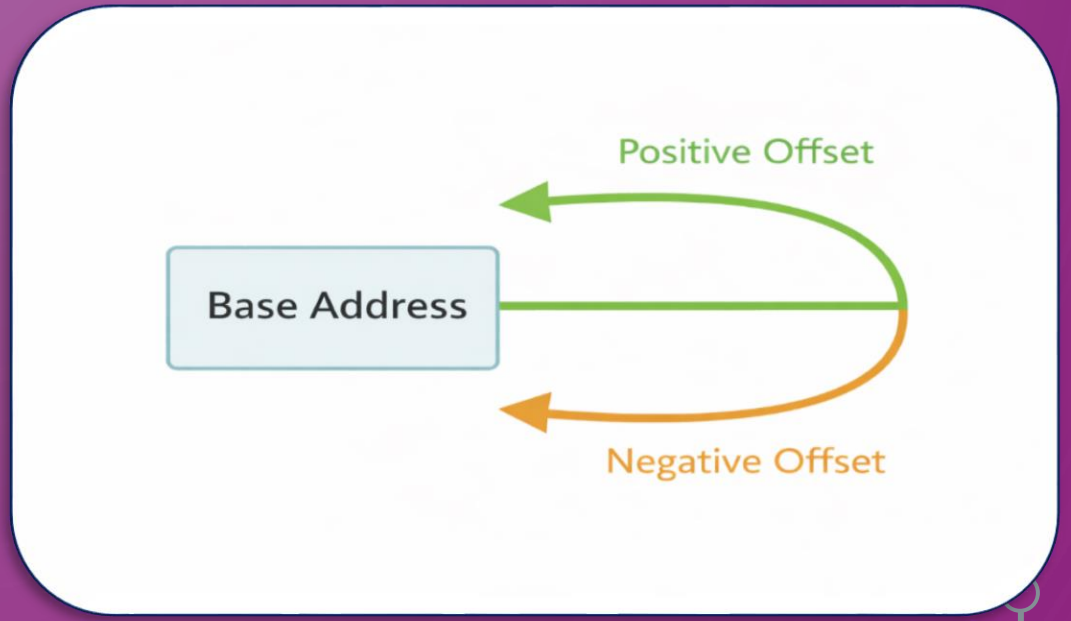
- Allows accessing a specific memory location by adding or subtracting an offset from a base address.
- **Example** :

- LDR R0, [R1, #4] ;

Load value from memory at address (R1 + 4)

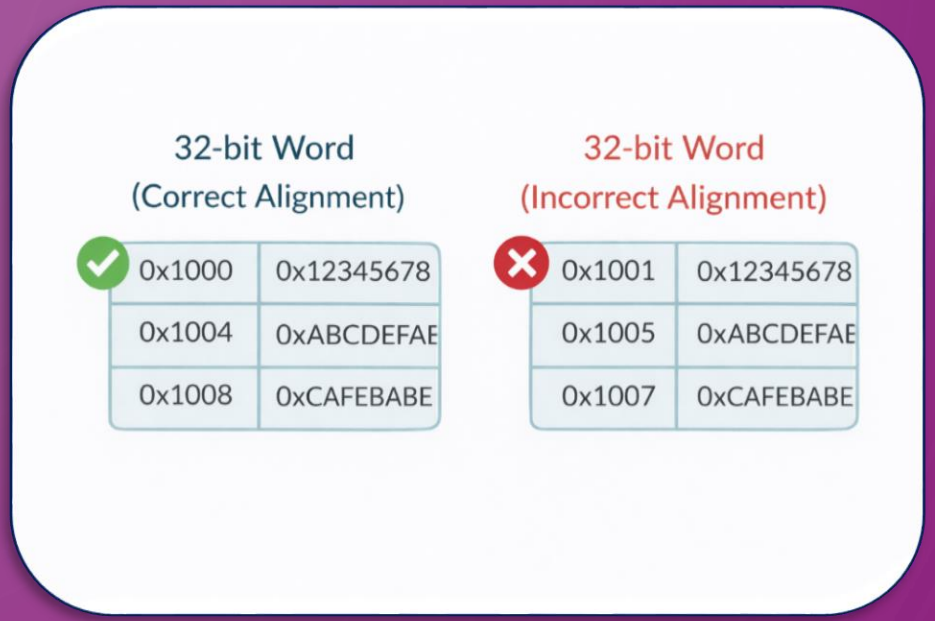
- STR R0, [R1, #-8] ;

Store value to memory at address (R1 - 8)



MEMORY ALIGNMENT

- Word (32 bits) → must be aligned at addresses divisible by 4
- Half-word (16 bits) → must be aligned at addresses divisible by 2
- Misaligned memory → can cause performance penalty or faults



- **Example** :

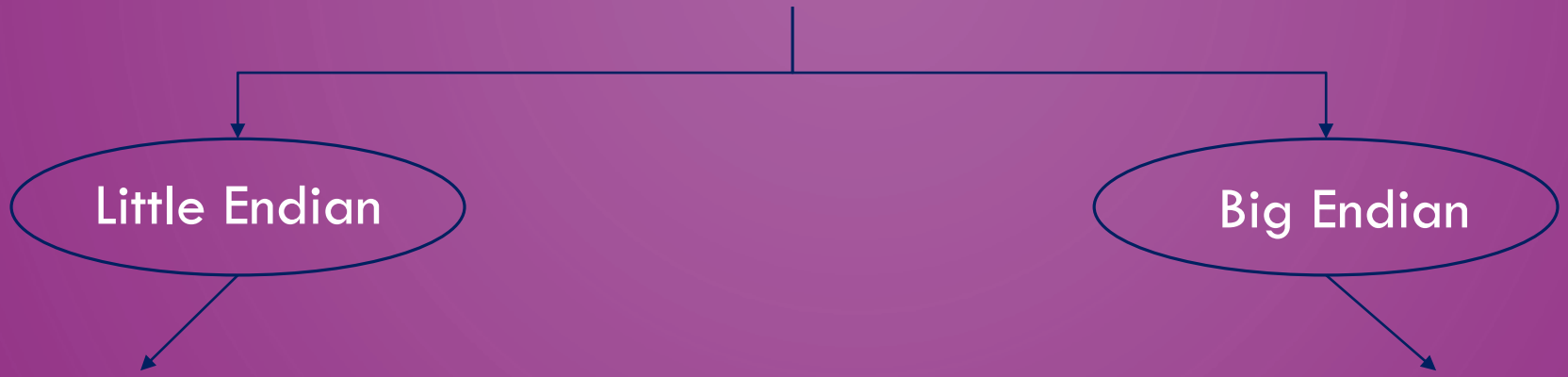
- .word 0x12345678 ;Address must be multiple of 4.

- .half 0xABCD ; Address must be multiple of 2.

ENDIANNESS (BYTE ORDER IN MEMORY)

- Endianness defines how bytes are arranged in memory.
- **Example Value:**
  0x12345678

➢ TYPES OF ENDIANNESS



-Lowest byte stored first.

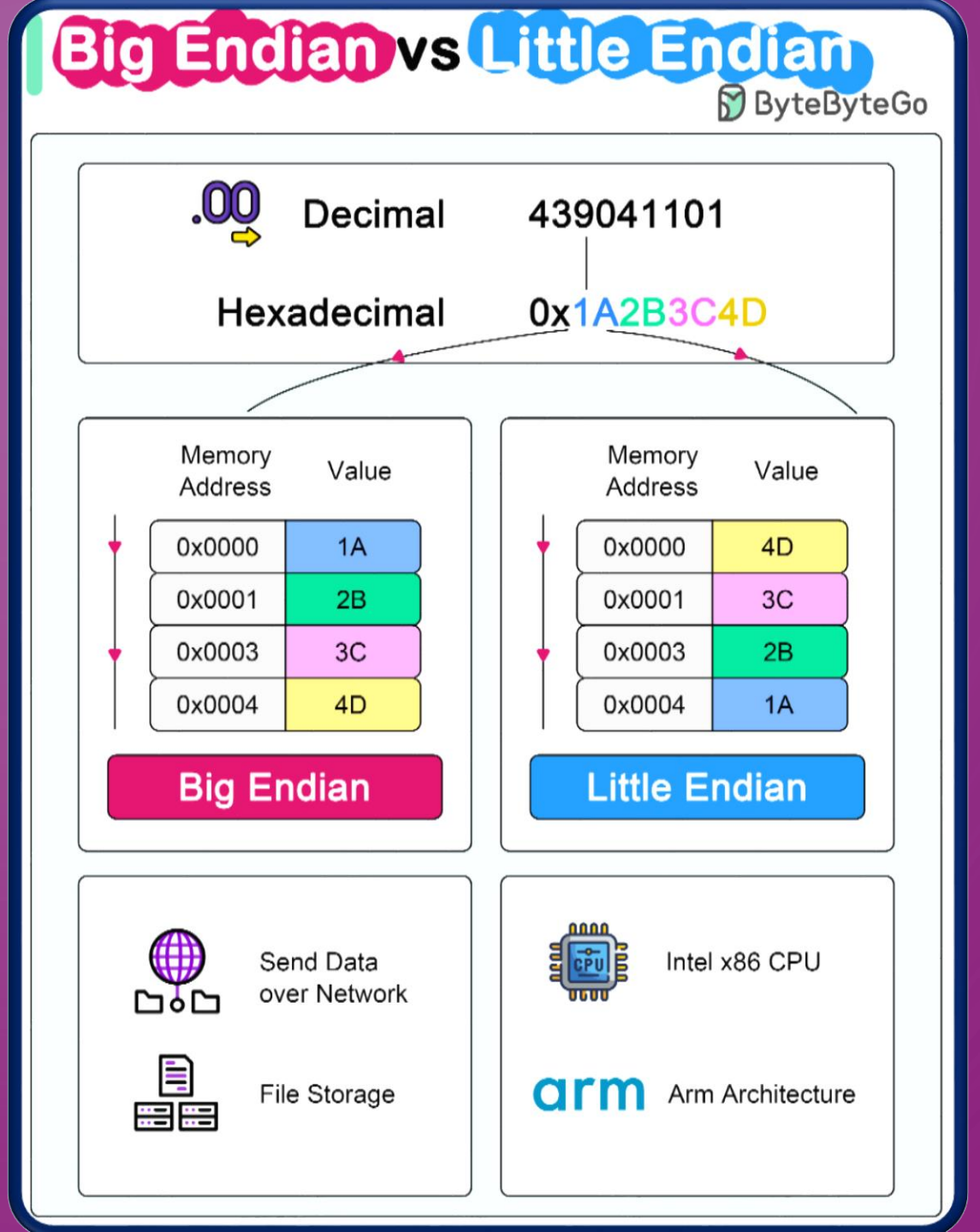
-Memory Layout:
12 34 56 78

-Highest byte stored first.

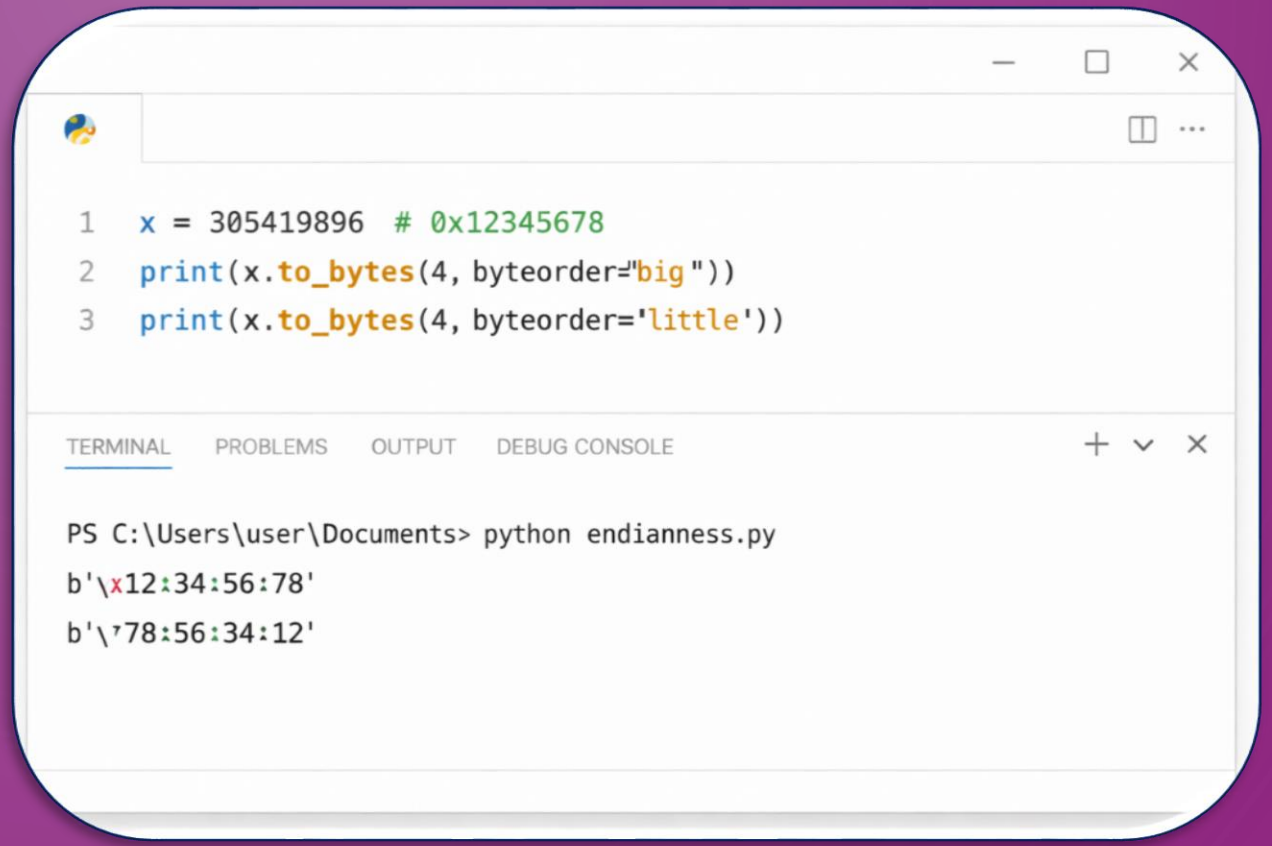
-Memory Layout:
78 56 34 12

- **Important when transferring data between systems.**

ENDIANNESS


Big Endian vs Little Endian
ByteByteGo
Decimal 439041101
Hexadecimal 0x1A2B3C4D
Memory Address
Value
0x0000 1A
0x0001 2B
0x0003 3C
0x0004 4D
Big Endian
Memory Address
Value
0x0000 4D
0x0001 3C
0x0003 2B
0x0004 1A
Little Endian
Send Data over Network
File Storage
Intel x86 CPU
Arm Architecture

ENDIANNESS


x = 305419896 # 0x12345678
print(x.to_bytes(4, byteorder="big"))
print(x.to_bytes(4, byteorder='little'))
TERMINAL
PROBLEMS
OUTPUT
DEBUG CONSOLE
PS C:\Users\user\Documents> python endianness.py
b'\x12:34:56:78'
b'\x78:56:34:12'

practical part

```
    .section .data
3 references
array:      .byte 10, 20, 30, 40       @ Original array (4 elements, 8-bit each)
1 reference
result:     .space 4                    @ Space for result array (4 bytes)
4 references
sum:        .word 0                     @ To store the sum

    .section .text
    .global _start

1 reference
_start:
    LDR R0, =array          @ Base address of original array
    LDR R1, =result         @ Base address of result array
    MOV R2, #4              @ Number of elements
    MOV R3, #0              @ Initialize sum = 0

2 references
loop:
    LDRB R4, [R0], #1       @ Post-indexing: load 1 byte from array, then increment R0
    ADD R5, R4, R4          @ Multiply value by 2
    STRB R5, [R1], #1       @ Post-indexing: store 1 byte into result, increment R1
    ADD R3, R3, R5          @ Add to sum
    SUBS R2, R2, #1         @ Decrement counter
    BNE loop                @ Repeat loop if counter not zero

    @ Store sum in memory
    LDR R6, =sum
    STR R3, [R6]            @ Store the 32-bit sum

1 reference
end:
    B end                   @ Infinite loop to stop program
```

result

```
Registers:
R0 = 0x20000000  ; base address of array
R1 = 0x20000004  ; base address of result
R2 = 0x0          ; counter
R3 = 0xC8         ; sum = 200
R4 = 0x28         ; last loaded byte (40 decimal)
R5 = 0x50         ; last result byte (80 decimal)
R6 = 0x20000008   ; address of sum

Memory:
array:  [10, 20, 30, 40]       ; original array
result: [20, 40, 60, 80]       ; modified array
sum: 200                        ; sum of result array
```

ASSIGNMENT2

Given:

- An array of 4 numbers, each 8-bit, stored in memory.

The program should:

1. Read each element using LDRB (because the elements are 8-bit).
2. Multiply each value by 2.
3. Store the result in a new array using STRB.
4. Calculate the sum of all modified elements using LDR.
5. Display the addresses and results (can be observed via a debugger).
6. Use **offset addressing** and **pre/post indexing**.
7. Take **endianness** into account when accessing memory.

THANKS

Prepared by Eng. Manar Ahmed